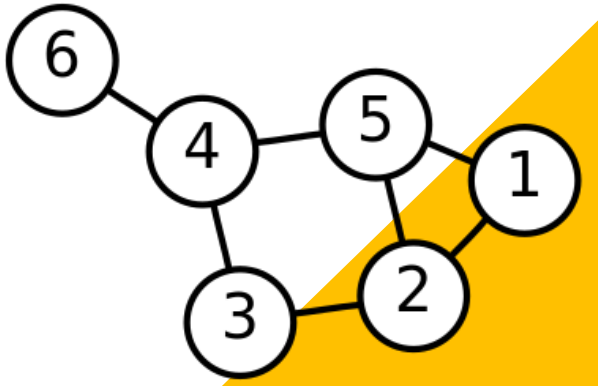
Grafos DFS-BFS

Grafos

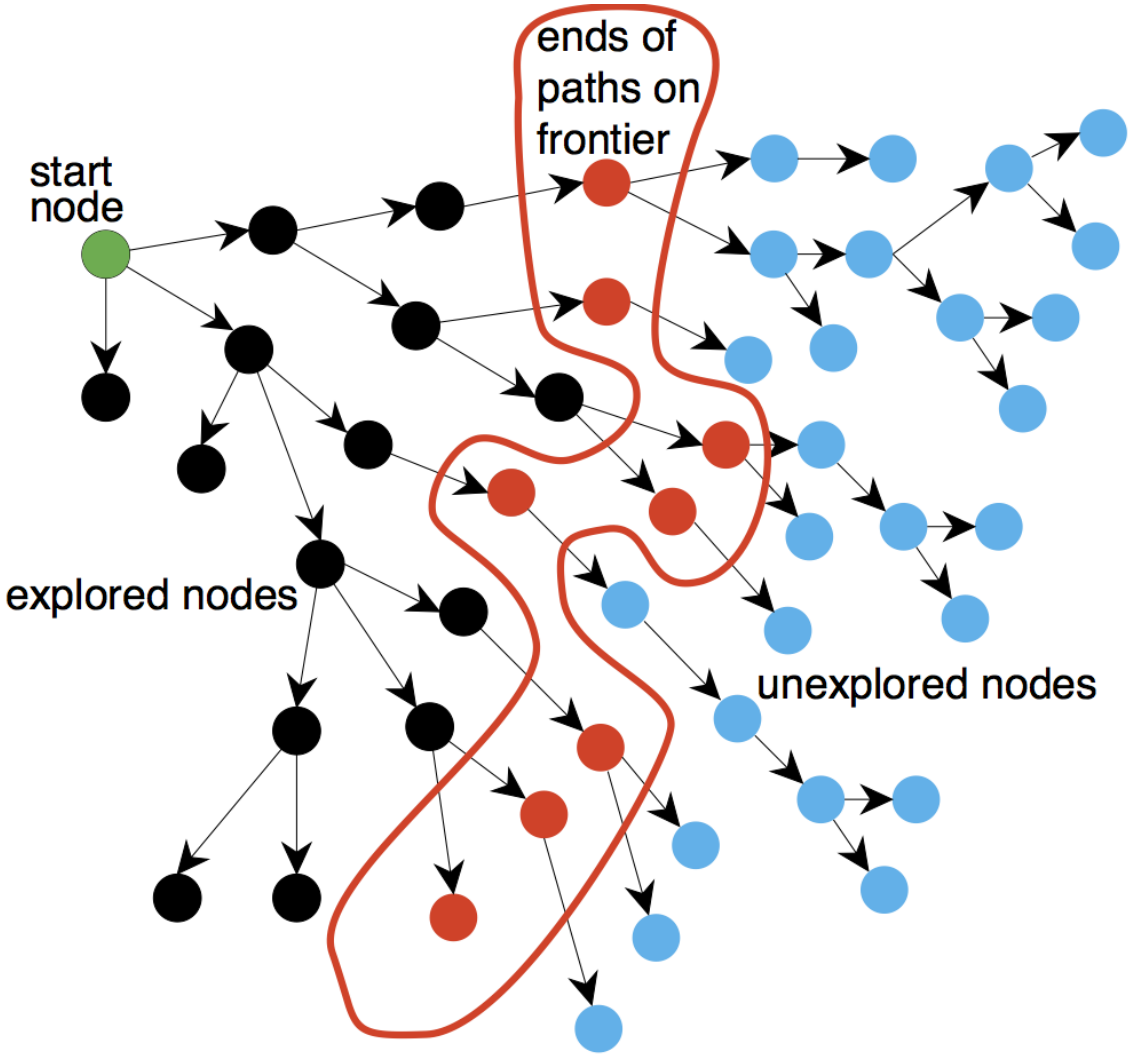
- **Grafo (G)**: Un grafo se define como un par ordenado (V, E), donde V es el conjunto de vértices o nodos, y E es el conjunto de aristas que conectan estos nodos. Los grafos pueden ser utilizados para modelar relaciones entre diferentes entidades.
- **Orden del Grafo**: El orden de un grafo G se refiere al número de vértices en el conjunto V. Se representa como $|V|$.
- **Grado de un Vértice**: El grado de un vértice (nodo) en un grafo es la cantidad de aristas que tienen ese vértice como uno de sus extremos. El grado de un vértice se utiliza para medir su conectividad en el grafo.
- **Bucle**: Un bucle es una arista que conecta un nodo consigo mismo. En otras palabras, es una arista donde el nodo de inicio y el nodo de llegada son el mismo.
- **Aristas Paralelas**: Dos o más aristas en un grafo se consideran paralelas si conectan el mismo par de vértices. Esto significa que existen múltiples caminos entre los mismos dos nodos.



Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

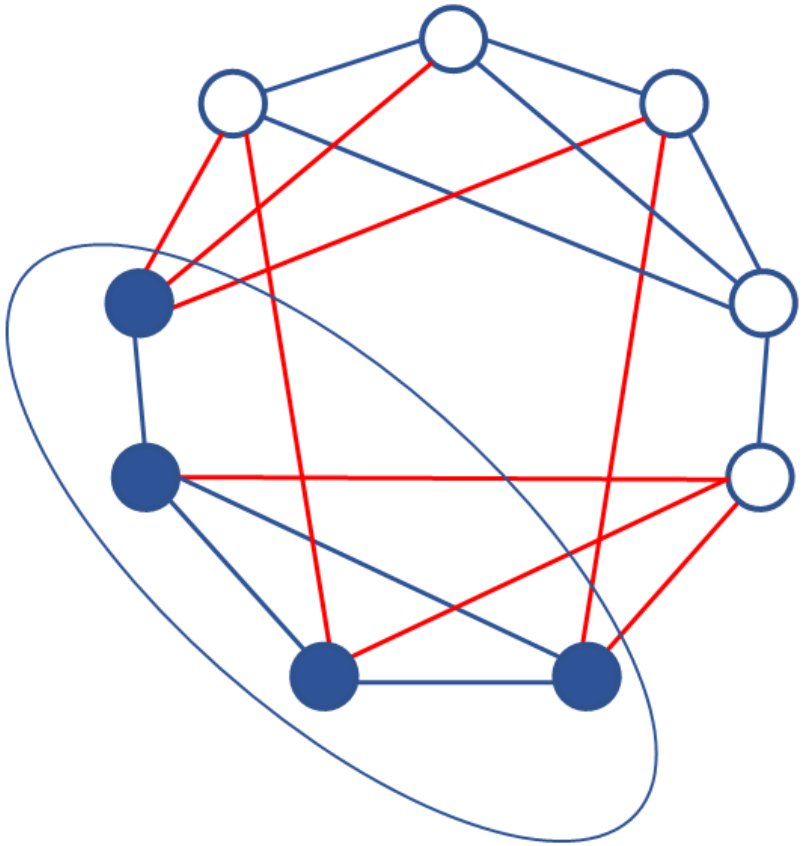
Algoritmos de Búsqueda en Grafos

- Los algoritmos de búsqueda en grafos resuelven problemas de navegación autónoma.
- Idea 2: Diferentes algoritmos ofrecen enfoques variados para la búsqueda de caminos óptimos.
- Idea 3: La elección del algoritmo depende de la información que mantienen y la eficiencia en la búsqueda.



Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Expansión

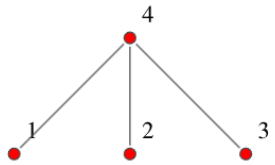


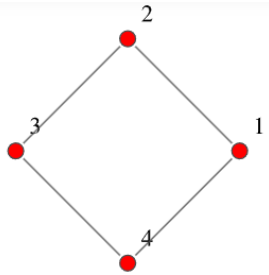
- Los algoritmos expanden la búsqueda desde el nodo inicial, explorando sus nodos vecinos.
- La expansión se realiza en forma de árbol, explorando nodos sucesores en cada nivel.
- La búsqueda se detiene al encontrar el nodo objetivo, optimizando el camino en términos de costo o pasos.

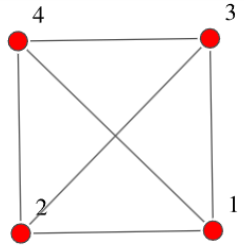
Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Notación y Lista de Nodos

- **Idea 1:** Las listas Abierta y Cerrada se utilizan para gestionar los nodos durante la búsqueda.
- **Idea 2:** La lista Abierta contiene nodos sin expandir, mientras que la lista Cerrada almacena nodos ya procesados.
- **Idea 3:** Cada nodo mantiene información sobre su predecesor, lo que facilita el seguimiento del camino óptimo.







$$\begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} \quad \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix} \quad \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

Step	Exploring	Open	Closed
1		$\{A\}$	$\{\}$
2	A	$\{B, C, D\}$	$\{A\}$
3	B	$\{E, F, C, D\}$	$\{A, B\}$
4	E	$\{G, F, C, D\}$	$\{A, B, E\}$
5	G	$\{F, C, D\}$	$\{A, B, E, G\}$
6	F	$\{C, D\}$	$\{A, B, E, G, F\}$
7	C	$\{D\}$	$\{A, B, E, G, F, C\}$
8	D	$\{H\}$	$\{A, B, E, G, F, C, D\}$
9	H	$\{\}$	$\{A, B, E, G, F, C, D, H\}$

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Pseudocodigo

```
Función BusquedaEnGrafo(Grafo, NodoInicial, NodoObjetivo):
    Crear una cola vacía
    Agregar NodoInicial a la cola
    Crear un conjunto para llevar un registro de nodos visitados
    Mientras la cola no esté vacía:
        Sacar un nodo de la cola
        Si este nodo es igual a NodoObjetivo:
            Se encontró el objetivo, devolver el camino
        Si el nodo no ha sido visitado:
            Marcar el nodo como visitado
            Para cada vecino del nodo:
                Si el vecino no ha sido visitado:
                    Agregar el vecino a la cola
                    Establecer el nodo actual como el predecesor del vecino
    # Si se llega aquí, no se encontró un camino
    Devolver "No se encontró un camino"

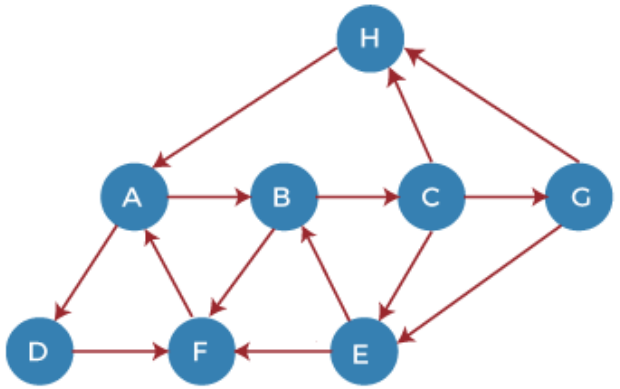
Función ReconstruirCamino(NodoInicial, NodoObjetivo):
    # Reconstruir el camino desde el NodoInicial al NodoObjetivo
    Camino = ListaVacia
    NodoActual = NodoObjetivo
    Mientras NodoActual no sea nulo:
        Agregar NodoActual al principio de Camino
        NodoActual = NodoActual.predecesor
    Devolver Camino
```

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Búsqueda en Profundidad DFS

- **Depth First Search**
- Principios de DFS: búsqueda en Profundidad, su enfoque en la profundidad de los nodos.
- Implementación en C*
- Aplicaciones de DFS: encontrar nodos conectados, ordenamiento topológico y resolución de puzzles.

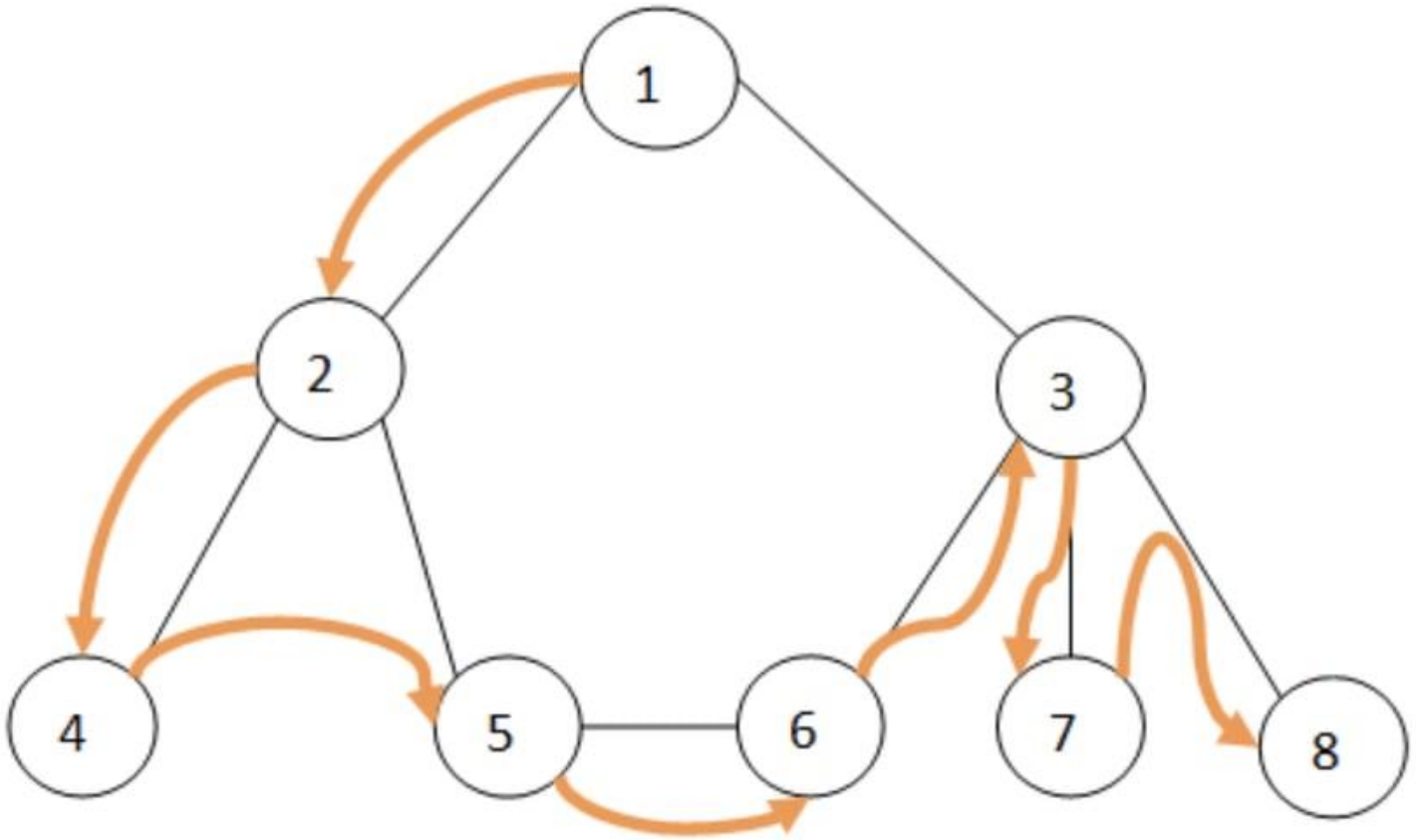


Adjacency Lists

A : B, D
B : C, F
C : E, G, H
G : E, H
E : B, F
F : A
D : F
H : A

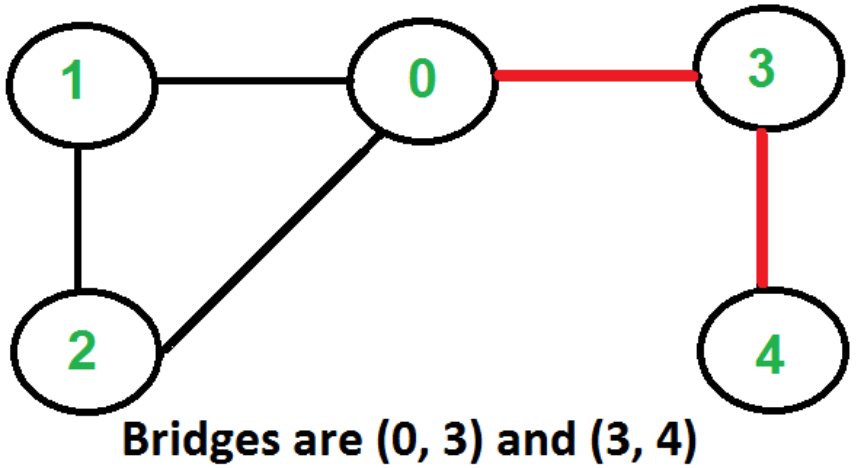
Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Búsqueda en Profundidad DFS



Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Búsqueda en Profundidad DFS



Bridges are (0, 3) and (3, 4)

- Puentes en Grafos: DFS se utiliza para encontrar puentes en grafos.
- Ordenamiento Topológico: DFS se aplica al ordenamiento topológico en un grafo acíclico dirigido.
- Importancia en IA: DFS se aplica en la resolución de problemas de inteligencia artificial.

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Búsqueda en Profundidad DFS

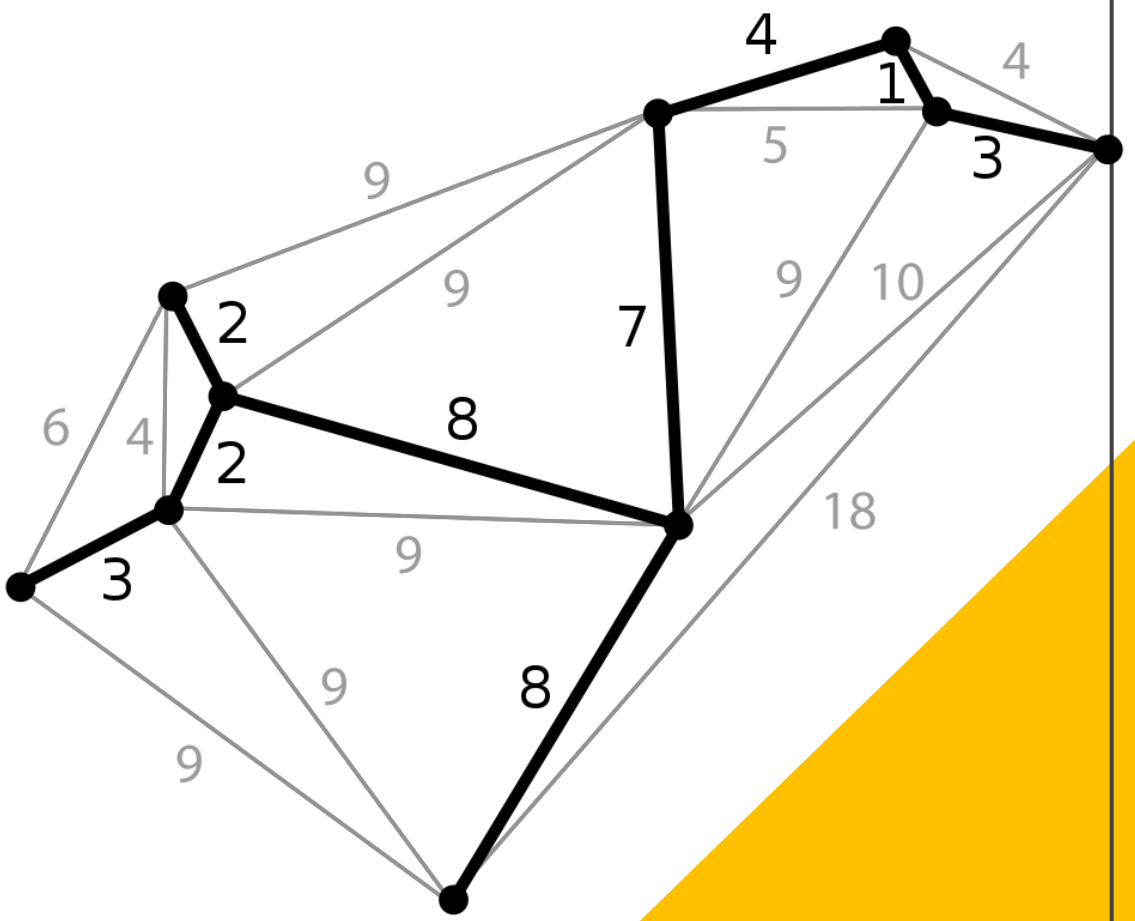
- **Completitud:** La DFS es completa solo cuando se utiliza en espacios de estado finitos. En este caso, todos los nodos se expandirán.
- **Optimalidad:** La DFS no garantiza la optimalidad, ya que puede encontrar soluciones más profundas antes de descubrir soluciones más óptimas en una rama no explorada.
- **Peor Caso Temporal:** En el peor caso, la complejidad temporal de la DFS es **O(b^m)**, donde **b** es el factor de ramificación promedio por nodo y **m** es la máxima profundidad del espacio de estados.
- **Complejidad Espacial:** La complejidad espacial de la DFS es **O(b^d)**, donde **b** es el factor de ramificación y **d** es la profundidad de la solución menos costosa. Cada nodo generado se mantiene en memoria, con la mayor cantidad de nodos almacenados en el nivel meta.
- **Limitaciones en Ambientes Infinitos:** La DFS no es adecuada para ambientes infinitos o espacios de estado con profundidades desconocidas.
- **Uso en Espacios Finitos:** En espacios de estado finitos, la DFS puede ser completa y eficiente en memoria, pero no garantiza la búsqueda de la solución óptima.

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

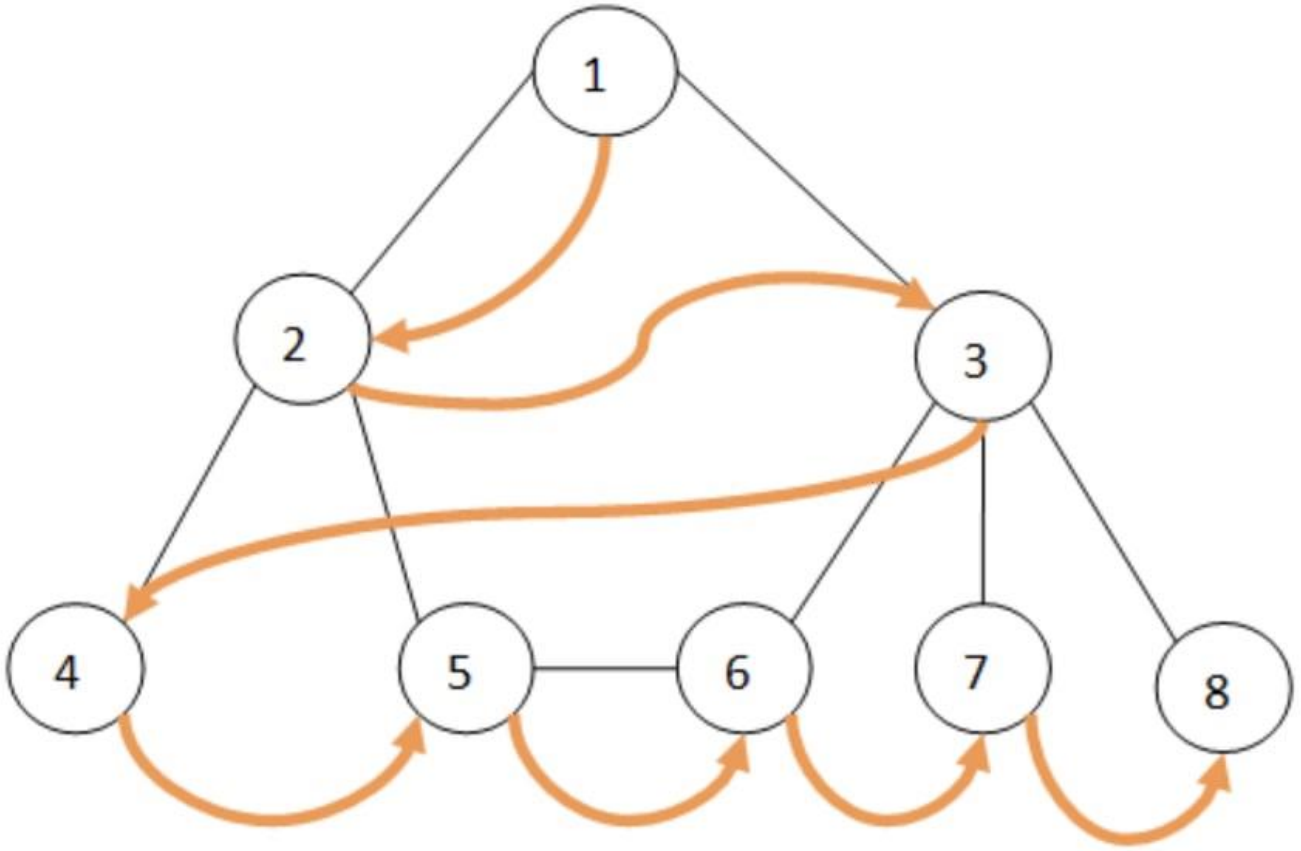
Principios de Búsqueda BFS

- **Breadth-First Search**
- Árbol de Expansión Mínima: BFS para encontrar el árbol de expansión mínima en grafos no ponderados.
- Uso en Web Crawlers: BFS en Web Crawlers para indexar sitios web.
- Relevancia en Sistemas de Navegación: BFS se utiliza en sistemas de navegación GPS.



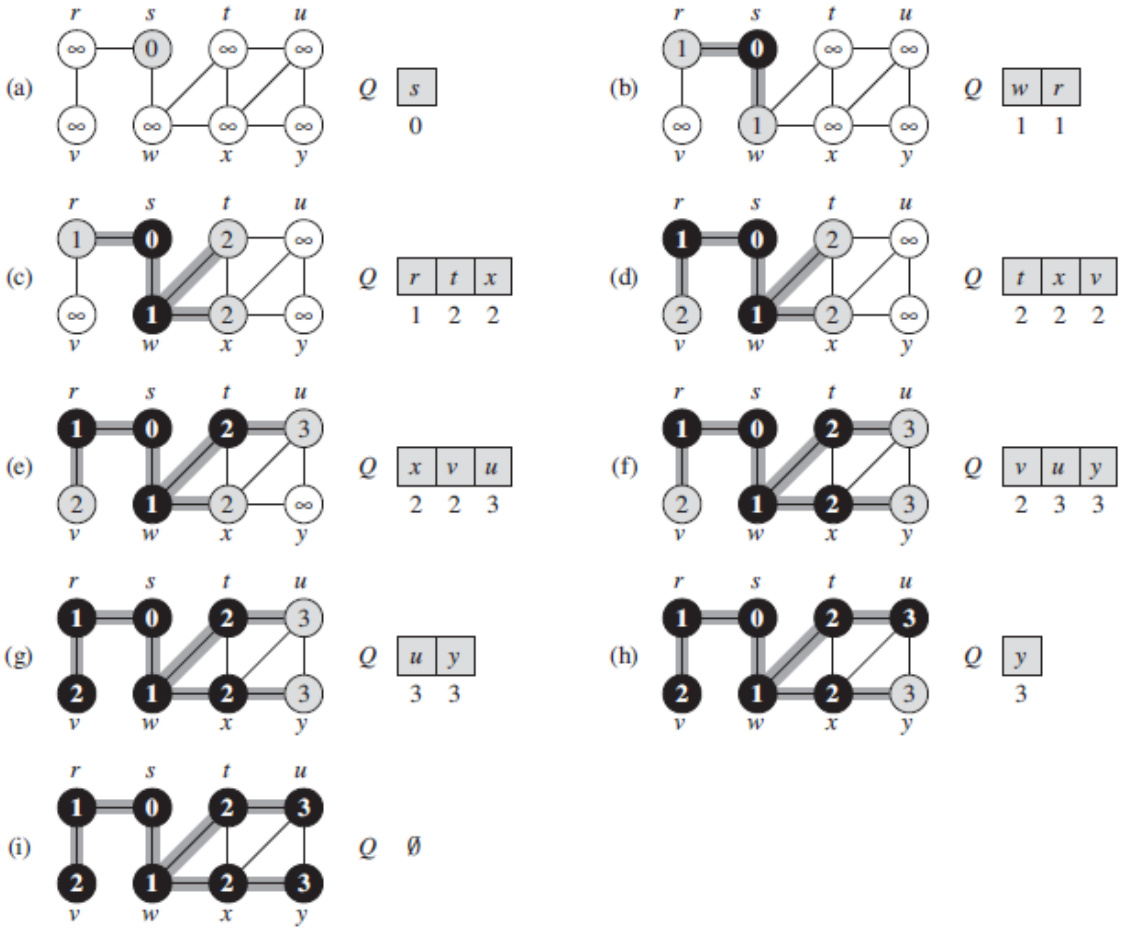
Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Principios de Búsqueda BFS



Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Principio de Búsqueda BFS



- Exploración en Capas: BFS avanza en capas desde el nodo de inicio.
- Cola de Nodos: Descripción de la cola y cómo se utiliza en la BFS.
- Marcado de Nodos: Explicación de cómo se evita la exploración infinita en grafos.

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Procedimiento de (BFS)

- **Dado un Vértice Fuente (s):** El algoritmo BFS comienza desde un vértice fuente **s**, que es el punto de partida para explorar el grafo.
- **Descubrimiento de Vértices Alcanzables:** BFS explora sistemáticamente los vértices del grafo para "descubrir" todos los vértices alcanzables desde **s**.
- **Cálculo de Distancias**: Calcula la distancia, que es el menor número de vértices necesarios para llegar desde s a todos los vértices alcanzables.
- **Árbol BF (Breadth-First Tree):** Produce un árbol BF con raíz en s que contiene todos los vértices alcanzables. Este árbol representa la estructura de exploración de BFS.
- **Camino Más Corto:** El camino desde s a cada vértice en este recorrido contiene el mínimo número de vértices, lo que lo convierte en el camino más corto, medido en términos de número de vértices.

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Caracterísitcas (BFS)

- **Expansión Uniforme:** BFS expande uniformemente la frontera entre lo descubierto y lo no descubierto. Llega a los nodos de distancia k solo después de haber llegado a todos los nodos a distancia k-1.
- **Eficiencia en la Búsqueda del Camino Más Corto:** Debido a su naturaleza, BFS es eficiente en la búsqueda del camino más corto en términos de número de vértices, lo que lo hace valioso en aplicaciones de rutas y distancias.
- **Uso del Nombre:** El nombre "Breadth-First Search" se deriva de su enfoque en expandir en anchura, explorando todos los vecinos de un vértice antes de avanzar a los vecinos de los vecinos.

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Complejidad BFS

- **Complejidad Computacional:** BFS se ejecuta en **O(|V| + |E|)**, donde **|V|** es el número de vértices y **|E|** es el número de aristas del grafo. En el peor caso, cada vértice y arista se visita.

- **Complejidad en Memoria:** La complejidad en memoria de BFS es O(|V|), dependiendo del número de vértices. Se necesita memoria adicional para marcar los nodos visitados y otras estructuras de datos auxiliares.

- **Importancia:** Estas complejidades son cruciales para comprender el rendimiento de BFS en términos de tiempo y memoria en problemas reales.

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Aplicaciones de BFS

- **Recorrido de Grafos:** Uso de la BFS para explorar y buscar caminos en grafos.
- **Resolución de Laberintos:** Ejemplo de aplicación en juegos y AI.
- **Redes Sociales:** BFS se utiliza en redes sociales en línea.
- **Optimización de Rutas:** Uso en sistemas de navegación y logística.
- **Inteligencia Artificial:** Aplicaciones en planificación y búsqueda en juegos.
- **Relevancia Continua:** Importancia de la BFS en la informática.

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl